

第五套题解汇总

第五套题解汇总

本题解用于整套讲评、集中查阅与打印。每道题均按照“题意整理 -> 部分分题解 -> 满分正解”的顺序编排，并保留各题已经采用的部分分梯度、最终部分分程序和满分程序。

目录

- T1 上下五千年
- T2 采购
- T3 整除序列
- T4 绿野仙踪 (green)

阅读说明

- 部分分按照分数从低到高讲解，并说明每一档的适用范围、复杂度和升级瓶颈。
- “最终交付的部分分程序”与对应题目的 `judge/some.cpp` 保持一致。
- 满分参考代码与对应题目的 `judge/std.cpp` 保持一致。

T1 上下五千年

题意整理

把一个日期的年份、月份和日期直接写成十进制并依次连接，中间不补前导零。若得到的字符串是回文串，则称这个日期为“回文时间”。

例如，2018 年 10 月 2 日连接后得到 2018102，它是回文串，所以这是一个回文时间。

日期必须是合法的公历日期：大月 31 天，小月 30 天；闰年二月 29 天，平年二月 28 天。闰年规则为：年份能被 400 整除，或者能被 4 整除但不能被 100 整除。

给定一个合法日期，请求出**严格晚于该日期**的第一个回文时间。

部分分题解

部分分总表

累计分值	年份范围	做法	复杂度
30	$y \leq 2200$	逐日模拟	$O(D)$
60	$y \leq 4000$	逐日模拟	$O(D)$
100	$y \leq 7000$	完整逐日模拟	$O(D)$

本题本身是一道日期模拟题，没有比逐日枚举更自然的算法层次。为了保留 T1 的基础定位，部分分只按照年份范围递进，而没有强行加入更高级知识点。

最终交付程序正确处理输入年份不超过 4000 的前六个测试点，实际获得 60 分。

为什么不能通过全部数据

输入年份大于 4000 时，部分分程序不继续搜索。

从部分分到满分

日期加一和回文判断不需要改变，只需取消年份限制，即可处理全部输入。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}

int days(int y, int m) {
    static int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    if (m == 2) {
        return d[m] + leap(y);
    }
    return d[m];
}

void nextDay(int &y, int &m, int &d) {
    d++;
    if (d > days(y, m)) {
        d = 1;
        m++;
    }
    if (m > 12) {
        m = 1;
        y++;
    }
}

bool palindromeDate(int y, int m, int d) {
    string s = to_string(y) + to_string(m) + to_string(d);
    string t = s;
    reverse(t.begin(), t.end());
```

```

    return s == t;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int y, m, d;
    cin >> y >> m >> d;

    // 该预处理范围覆盖年份不超过 4000 的累计 60% 数据。
    if (y > 4000) {
        cout << "0 0 0\n";
        return 0;
    }

    do {
        nextDay(y, m, d);
    } while (!palindromeDate(y, m, d));
    cout << y << ' ' << m << ' ' << d << '\n';
    return 0;
}

```

满分正解

满分算法

实现公历日期加一：

1. 日期增加 1；
2. 超过当月天数时进入下个月；
3. 月份超过 12 时进入下一年；
4. 将 year、month、day 用 to_string 连接，判断是否回文。

找到第一个回文日期后输出。

正确性说明

算法从输入日期的严格后继开始，按照时间先后顺序逐日检查。所有检查日期均合法；第一个通过回文判断的日期显然是严格晚于输入的最近回文时间。

复杂度分析

设答案与输入相差 D 天，时间复杂度为 $O(D)$ ，空间复杂度为 $O(1)$ 。在给定年份范围内，实际需要检查的日期数量很小。

边界与易错点

- 若输入本身是回文时间，仍然必须寻找下一个；
- 月和日不能补成两位；
- 世纪年必须能被 400 整除才是闰年；
- 输出年份可能略大于 7000，因为限制只约束输入日期。

参考代码

```

#include<bits/stdc++.h>
using namespace std;

bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}

int days(int y, int m) {
    static int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    if (m == 2) {
        return d[m] + leap(y);
    }
    return d[m];
}

void nextDay(int &y, int &m, int &d) {

```

```

    d++;
    if (d > days(y, m)) {
        d = 1;
        m++;
    }
    if (m > 12) {
        m = 1;
        y++;
    }
}

bool palindromeDate(int y, int m, int d) {
    string s = to_string(y) + to_string(m) + to_string(d);
    string t = s;
    reverse(t.begin(), t.end());
    return s == t;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int y, m, d;
    cin >> y >> m >> d;
    do {
        nextDay(y, m, d);
    } while (!palindromeDate(y, m, d));

    cout << y << ' ' << m << ' ' << d << '\n';
    return 0;
}

```

T2 采购

题意整理

牛牛要从 n 种商品中采购，每种商品只有一件。第 i 件商品的体积为 w_i ，价值为 val_i 。所选商品总体积不能超过 m 。

若选择商品的总价值为 VAL 、商品数量为 $COUNT$ ，定义本次选择的收益为

$$f(k) = VAL - k \times COUNT$$

。

牛牛可以一件商品都不选择。求收益的最大值。

部分分题解

部分分总表

分值	特殊限制	做法	复杂度
20	$n \leq 20$	枚举全部子集	$O(2^n)$
30	$m \leq 300$	二维 0/1 背包	$O(nm)$ 时间, $O(nm)$ 空间
100	无额外限制	一维倒序背包	$O(nm)$ 时间, $O(m)$ 空间

小 n 枚举

枚举每件商品选或不选，统计体积与收益，保留合法方案最大值。

小容量二维 DP

当 $m \leq 300$ 时，可以定义 $dp[i][j]$ 表示只考虑前 i 件商品、容量不超过 j 的最大收益，直接进行二维转移。最终交付程序组合两类做法，实际通过测试点 1 ~ 5，获得 50 分。

为什么不能通过全部数据

一般数据不满足 $n \leq 20$ 或 $m \leq 300$ 。虽然二维 DP 在部分较大数据上也可能运行，但不属于公开保证范围。

从部分分到满分

观察到第 i 层只依赖第 $i - 1$ 层，把容量倒序枚举即可压缩成一维数组，稳定处理 $m \leq 3000$ 。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    long long k;
    cin >> n >> m >> k;
    vector<int> w(n);
    vector<long long> v(n);
    for (int i = 0; i < n; i++) {
        cin >> w[i] >> v[i];
        v[i] -= k;
    }

    if (n <= 20) {
        long long ans = 0;
        for (int mask = 0; mask < (1 << n); mask++) {
            int sumW = 0;
            long long sumV = 0;
            for (int i = 0; i < n; i++) {
                if (mask >> i & 1) {
                    sumW += w[i];

```

```

        sumV += v[i];
    }
}
if (sumW <= m) {
    ans = max(ans, sumV);
}
}
cout << ans << '\n';
return 0;
}

if (m <= 300) {
    // 使用二维 0/1 背包，便于初学者理解“前 i 个物品”的含义。
    vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            dp[i][j] = dp[i - 1][j];
            if (j >= w[i - 1]) {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - w[i - 1]] + v[i - 1]);
            }
        }
    }
    cout << *max_element(dp[n].begin(), dp[n].end()) << '\n';
    return 0;
}

cout << 0 << '\n';
return 0;
}

```

满分正解

题意转化

每选择一件商品，目标函数增加

$$val_i - k$$

。

因此把每件商品的新价值定义为 $v_i = val_i - k$ ，题目就是容量为 m 的 0/1 背包。允许不选任何商品，所以答案至少为 0。

满分算法

定义 $dp[j]$ 为总体积不超过 j 时的最大收益。对每件商品按容量从大到小转移：

$$dp[j] = \max(dp[j], dp[j - w_i] + val_i - k)$$

。

正确性说明

目标函数可以按商品拆分为每件被选商品的独立贡献 $val_i - k$ 。每件商品至多选择一次，体积和不超过 m ，完全符合 0/1 背包模型。倒序枚举容量保证同一件商品不会被重复使用。

复杂度分析

时间复杂度 $O(nm)$ ，空间复杂度 $O(m)$ 。

边界与易错点

- 可以不选择商品，不能把答案初始化为负无穷；
- $val_i - k$ 可能为负；
- 容量必须倒序枚举；
- 目标值建议使用 `long long`。

参考代码

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    long long k;
    cin >> n >> m >> k;
    vector<long long> dp(m + 1, 0);

    for (int i = 1; i <= n; i++) {
        int w;
        long long value;
        cin >> w >> value;
        long long gain = value - k;
        for (int j = m; j >= w; j--) {
            dp[j] = max(dp[j], dp[j - w] + gain);
        }
    }

    cout << *max_element(dp.begin(), dp.end()) << '\n';
    return 0;
}
```

T3 整除序列

题意整理

给定两个整数 L, R ，需要构造一个严格递增的整数序列

$$a_1 < a_2 < \dots < a_S,$$

满足：

- $L \leq a_i \leq R$;
- 对任意 $i < j$ ，都有 $a_j \bmod a_i = 0$ 。

求能够构造出的最大长度 S ，以及长度恰好为 S 的不同序列数量。

两个序列不同，当且仅当长度不同或至少一个位置上的数不同。

本题包含多组询问，方案数需要输出完整十进制表示。

部分分题解

部分分总表

分值	特殊限制	做法	复杂度
20	$R - L \leq 20$	枚举区间中所有数对进行 DP	$O((R - L + 1)^2)$
30	$R \leq 2000$	枚举所有 $x < y$ ，判断 $x \mid y$	$O((R - L + 1)^2)$
100	无额外限制	只枚举每个数的倍数	$O(R \log R)$

二次方 DP

区间按数值天然有序。枚举所有 $i < j$ ，若 $(L + j) \bmod (L + i) = 0$ ，就进行最长路和方案数转移。

最终交付程序在区间长度不超过 21 或 $R \leq 2000$ 时使用该方法，实际通过测试点 1 ~ 5，获得 50 分。

为什么不能通过全部数据

区间长度接近 10^5 时，二次方枚举需要约 10^{10} 次判断。

从部分分到满分

对于固定的 x ，无需检查所有更大的 y ，只需枚举 $2x, 3x, \dots$ 。这样转移数量降为调和级数规模。

最终交付的部分分程序

```
#include <bits/stdc++.h>
#include <boost/multiprecision/cpp_int.hpp>
using namespace std;
using boost::multiprecision::cpp_int;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int L, R;
        cin >> L >> R;
        int n = R - L + 1;
        if (!(n <= 21 || R <= 2000)) {
            cout << "0 0\n";
            continue;
        }

        vector<int> len(n, 1);
        vector<cpp_int> ways(n, 1);
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
```

```

        int x = L + i;
        int y = L + j;
        if (y % x == 0) {
            int candidate = len[i] + 1;
            if (candidate > len[j]) {
                len[j] = candidate;
                ways[j] = ways[i];
            } else if (candidate == len[j]) {
                ways[j] += ways[i];
            }
        }
    }
}

int bestLen = 0;
cpp_int answer = 0;
for (int i = 0; i < n; i++) {
    if (len[i] > bestLen) {
        bestLen = len[i];
        answer = ways[i];
    } else if (len[i] == bestLen) {
        answer += ways[i];
    }
}
cout << bestLen << ' ' << answer << '\n';
}
return 0;
}

```

满分正解

关键观察

因为序列严格递增，若相邻两项满足后一项是前一项的倍数，则更早的项也一定整除后一项。因此只需要维护相邻转移。

把区间中的每个整数看作一个点。若 $x < y$ 且 $x \mid y$ ，就可以从 x 转移到 y 。所有边都从小数指向大数，构成 DAG。

满分算法

定义：

- $\text{len}[x]$ ：以 x 结尾的最长合法序列长度；
- $\text{ways}[x]$ ：达到该长度的方案数。

按照 $x = L, L + 1, \dots, R$ 处理，并枚举 x 的所有倍数 $2x, 3x, \dots$ 进行最长路计数转移。方案数使用 `cpp_int` 保存。

正确性说明

所有合法序列的最后一步一定从某个能整除末项的较小数转移而来；算法枚举了每个 x 在区间内的全部更大倍数，因此覆盖所有合法转移。按数值递增处理保证转移来源已经完成，标准 DAG 最长路计数即可得到每个终点的最长长度与方案数。最后汇总全局最大长度的终点。

复杂度分析

时间复杂度为

$$O\left(\sum_{x=L}^R \frac{R}{x}\right),$$

最坏约为 $O(R \log R)$ ；空间复杂度 $O(R)$ ，另加大整数存储。

边界与易错点

- 长度为 1 的每个单元序列都是一种方案；
- 方案数可能很大，不能使用普通整数假设；
- 只有严格更大的倍数才能转移；

- 多组询问需要重新初始化状态。

参考代码

```
#include<bits/stdc++.h>
#include<boost/multiprecision/cpp_int.hpp>
using namespace std;
using boost::multiprecision::cpp_int;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int L, R;
        cin >> L >> R;
        vector<int> len(R + 1, 1);
        vector<cpp_int> ways(R + 1, 1);

        for (int x = L; x <= R; x++) {
            for (int y = x + x; y <= R; y += x) {
                int candidate = len[x] + 1;
                if (candidate > len[y]) {
                    len[y] = candidate;
                    ways[y] = ways[x];
                } else if (candidate == len[y]) {
                    ways[y] += ways[x];
                }
            }
        }

        int bestLen = 0;
        cpp_int answer = 0;
        for (int x = L; x <= R; x++) {
            if (len[x] > bestLen) {
                bestLen = len[x];
                answer = ways[x];
            } else if (len[x] == bestLen) {
                answer += ways[x];
            }
        }
        cout << bestLen << ' ' << answer << '\n';
    }
    return 0;
}
```

T4 绿野仙踪 (green)

题意整理

公园中有一块 $n \times m$ 的矩形方格场地，方格坐标满足

$$0 \leq x < n, \quad 0 \leq y < m$$

。

公园准备以 k 个给定方格 (x_i, y_i) 为中心种植大小相同的正方形花圃。给定正整数 r ，每个花圃边长为 $2r - 1$ ，覆盖所有满足

$$|x - x_i| \leq r - 1, \quad |y - y_i| \leq r - 1$$

的场内方格。超出场地的部分忽略，不同花圃的重叠方格只计算一次。

给定游园起点 (s_x, s_y) 和终点 (e_x, e_y) 。游客每一步可以向上、下、左、右移动到相邻方格，不能进入花圃覆盖的方格。花圃也不能覆盖起点或终点。

请选择统一的 r ，使起点到终点至少存在一条合法路径，并最大化所有花圃覆盖的方格总数。若不存在任何合法的正整数 r ，输出 0。

部分分题解

部分分总表

测试点	特殊性质	分值	可行做法
1 ~ 2	小网格且 $k \leq 10$	10	二分 r ，直接标记或二维差分后 BFS
3 ~ 4	$k = 1$	10	单矩形标记后 BFS
5 ~ 6	$k \leq 10$	10	二分、矩形标记与 BFS
7 ~ 8	起终点相邻	10	只要两端未覆盖就天然连通，再统计面积
9 ~ 10	两个对角	10	二分与 BFS
11 ~ 20	一般数据	50	二维差分加 BFS

这些子任务是若干独立特殊性质。最终交付程序判断是否满足 $k \leq 10$ 、起终点相邻或位于两个指定对角；满足时运行正确的二分、二维差分和 BFS，实际通过测试点 1 ~ 10，获得 50 分。

为什么不能通过全部数据

一般测试点中 $k > 10$ ，起终点既不相邻也不是规定的两个对角，部分分程序不处理。

从部分分到满分

二分、二维差分和 BFS 本身已经适用于一般情况。取消对子任务性质的判断即可处理全部数据。

最终交付的部分分程序

```
#include<bits/stdc++.h>
using namespace std;

struct Solver {
    int n, m, k;
    int sx, sy, ex, ey;
    vector<pair<int,int>> center;

    bool check(int r, long long *area = nullptr) {
        if (r == 0) {
            if (*area) *area = 0;
            return true;
        }
    }
}
```

```

int w = m + 1;
vector<int> diff((n + 1) * (m + 1), 0);
auto addRect = [&](int x1, int y1, int x2, int y2) {
    diff[x1 * w + y1]++;
    diff[(x2 + 1) * w + y1]--;
    diff[x1 * w + (y2 + 1)]--;
    diff[(x2 + 1) * w + (y2 + 1)]++;
};

for (const auto &p : center) {
    int x = p.first;
    int y = p.second;
    int x1 = max(0, x - r + 1);
    int x2 = min(n - 1, x + r - 1);
    int y1 = max(0, y - r + 1);
    int y2 = min(m - 1, y + r - 1);
    addRect(x1, y1, x2, y2);
}

vector<unsigned char> blocked(n * m, 0);
long long cnt = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        int id = i * w + j;
        if (i > 0) diff[id] += diff[(i - 1) * w + j];
        if (j > 0) diff[id] += diff[i * w + j - 1];
        if (i > 0 && j > 0) diff[id] -= diff[(i - 1) * w + j - 1];
        if (diff[id] > 0) {
            blocked[i * m + j] = 1;
            cnt++;
        }
    }
}
if (area) *area = cnt;

int start = sx * m + sy;
int target = ex * m + ey;
if (blocked[start] || blocked[target]) {
    return false;
}

vector<unsigned char> vis(n * m, 0);
vector<int> q(n * m);
int head = 0, tail = 0;
q[tail++] = start;
vis[start] = 1;
const int dx[4] = {1, -1, 0, 0};
const int dy[4] = {0, 0, 1, -1};

while (head < tail) {
    int id = q[head++];
    if (id == target) {
        return true;
    }
    int x = id / m;
    int y = id % m;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d];
        int ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        int nid = nx * m + ny;
        if (!blocked[nid] && !vis[nid]) {
            vis[nid] = 1;
            q[tail++] = nid;
        }
    }
}
return false;
}

long long solve() {
    int lo = 0;
    int hi = max(n, m) + 2;

```

```

        while (lo + 1 < hi) {
            int mid = (lo + hi) / 2;
            if (check(mid)) {
                lo = mid;
            } else {
                hi = mid;
            }
        }
        long long area = 0;
        check(lo, &area);
        return area;
    }
};

int main() {
    freopen("green.in", "r", stdin);
    freopen("green.out", "w", stdout);

    Solver solver;
    cin >> solver.n >> solver.m >> solver.k;
    cin >> solver.sx >> solver.sy;
    cin >> solver.ex >> solver.ey;
    solver.center.resize(solver.k);
    for (auto &p : solver.center) {
        cin >> p.first >> p.second;
    }

    bool adjacent = abs(solver.sx - solver.ex) + abs(solver.sy - solver.ey) == 1;
    bool corners = solver.sx == 0 && solver.sy == 0 &&
        solver.ex == solver.n - 1 && solver.ey == solver.m - 1;

    // 覆盖原题测试点 1~10: k<=10、起终点相邻或位于两个对角。
    if (solver.k <= 10 || adjacent || corners) {
        cout << solver.solve() << '\n';
    } else {
        cout << 0 << '\n';
    }
    return 0;
}

```

满分正解

关键观察

随着 r 增大，每个花圃覆盖范围只会扩大，场地中的可行道路只会减少。因此“半径 r 是否可行”具有单调性，可以二分最大可行 r 。

对固定 r ，每个花圃对应一个经过边界裁剪的矩形。使用二维差分，可以在 $O(k + nm)$ 时间内求出所有被至少一个花圃覆盖的方格。

满分算法

1. 二分 r ;
2. 对每个中心向二维差分数组加入覆盖矩形;
3. 前缀还原阻塞方格，并统计并集面积;
4. 若起点或终点被阻塞则不可行;
5. 在未阻塞方格中进行四方向 BFS，判断终点是否可达;
6. 对最大可行 r 再计算一次覆盖面积。

内部把 $r = 0$ 作为“没有花圃”的二分下界，因此无正整数方案时面积为 0。

正确性说明

二维差分精确标记了所有花圃矩形的并集；BFS 精确判断四联通路径。由于覆盖集合随 r 单调扩大，可行性单调不减，二分得到最大的可行 r 。同一中心的覆盖范围随 r 增大，因此最大可行 r 也使并集面积最大。

复杂度分析

每次检查为 $O(nm + k)$ ，二分次数为 $O(\log \max(n, m))$ 。总时间复杂度

$$O((nm + k) \log \max(n, m)),$$

空间复杂度 $O(nm + k)$ 。

边界与易错点

- 花圃越界部分需要裁剪;
- 重叠位置只能计算一次;
- 起点和终点不能被覆盖;
- 路径使用上下左右四个方向;
- 可能不存在任何正整数 r , 此时输出 0。

参考代码

```
#include <bits/stdc++.h>
using namespace std;

struct Solver {
    int n, m, k;
    int sx, sy, ex, ey;
    vector<pair<int, int>> center;

    bool check(int r, long long *area = nullptr) {
        if (r == 0) {
            if (area) *area = 0;
            return true;
        }

        int w = m + 1;
        vector<int> diff((n + 1) * (m + 1), 0);
        auto addRect = [&](int x1, int y1, int x2, int y2) {
            diff[x1 * w + y1]++;
            diff[(x2 + 1) * w + y1]--;
            diff[x1 * w + (y2 + 1)]--;
            diff[(x2 + 1) * w + (y2 + 1)]++;
        };

        for (const auto &p : center) {
            int x = p.first;
            int y = p.second;
            int x1 = max(0, x - r + 1);
            int x2 = min(n - 1, x + r - 1);
            int y1 = max(0, y - r + 1);
            int y2 = min(m - 1, y + r - 1);
            addRect(x1, y1, x2, y2);
        }

        vector<unsigned char> blocked(n * m, 0);
        long long cnt = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                int id = i * w + j;
                if (i > 0) diff[id] += diff[(i - 1) * w + j];
                if (j > 0) diff[id] += diff[i * w + j - 1];
                if (i > 0 && j > 0) diff[id] -= diff[(i - 1) * w + j - 1];
                if (diff[id] > 0) {
                    blocked[i * m + j] = 1;
                    cnt++;
                }
            }
        }
        if (area) *area = cnt;

        int start = sx * m + sy;
        int target = ex * m + ey;
        if (blocked[start] || blocked[target]) {
            return false;
        }

        vector<unsigned char> vis(n * m, 0);
        vector<int> q(n * m);
        int head = 0, tail = 0;
        q[tail++] = start;
        vis[start] = 1;
        const int dx[4] = {1, -1, 0, 0};
```

```

const int dy[4] = {0, 0, 1, -1};

while (head < tail) {
    int id = q[head++];
    if (id == target) {
        return true;
    }
    int x = id / m;
    int y = id % m;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d];
        int ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        int nid = nx * m + ny;
        if (!blocked[nid] && !vis[nid]) {
            vis[nid] = 1;
            q[tail++] = nid;
        }
    }
}
return false;
}

long long solve() {
    int lo = 0;
    int hi = max(n, m) + 2;
    while (lo + 1 < hi) {
        int mid = (lo + hi) / 2;
        if (check(mid)) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    long long area = 0;
    check(lo, &area);
    return area;
}

};

int main() {
    freopen("green.in", "r", stdin);
    freopen("green.out", "w", stdout);

    Solver solver;
    cin >> solver.n >> solver.m >> solver.k;
    cin >> solver.sx >> solver.sy;
    cin >> solver.ex >> solver.ey;
    solver.center.resize(solver.k);
    for (auto &p : solver.center) {
        cin >> p.first >> p.second;
    }
    cout << solver.solve() << '\n';
    return 0;
}

```